

Scaling Training Across Multi-GPU Nodes

Optimizing performance, training larger models, and maximizing hardware efficiency through advanced parallelism techniques and frameworks



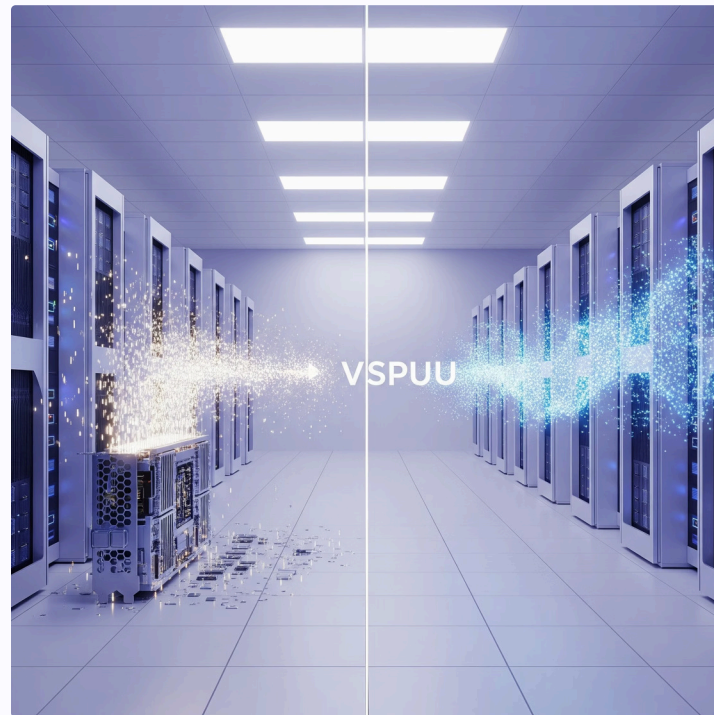
Why Multi-GPU Training?

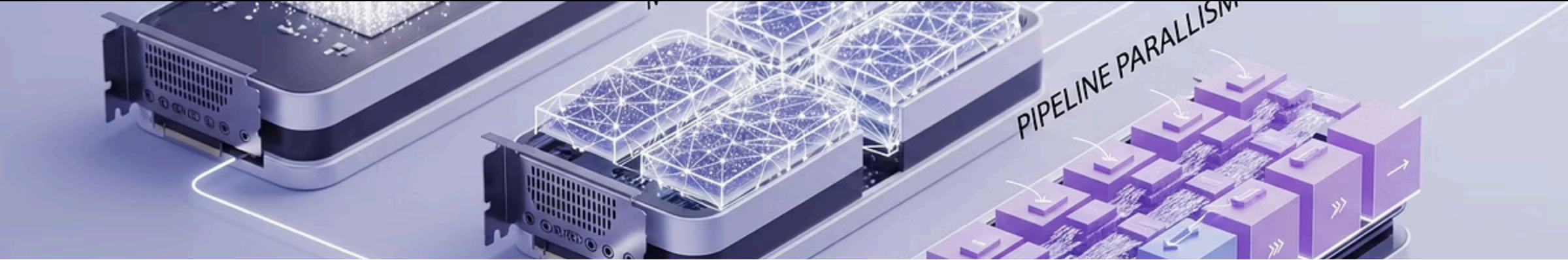
Single GPU limitations:

- Memory constraints (24-80GB per device)
- Extended training periods (weeks) with large datasets

Key advantages:

- Significantly higher throughput
- Ability to train massive models (LLMs, ViTs)
- Optimal hardware resource utilization





Strategies for Scaling

1

Data Parallelism

Model replicated on each GPU

Data batches distributed across devices

Gradients synchronized via all-reduce operations

Implementations: PyTorch DDP, Horovod

2

Model Parallelism

Model layers/tensors split across GPUs

Essential for extremely large models that exceed single-GPU memory

Implementations: Megatron-LM, DeepSpeed

3

Pipeline Parallelism

Model divided into sequential stages across GPUs

Mini-batches processed in pipeline fashion to maximize throughput

Reduces activation memory requirements

Hybrid Approaches

Tensor Parallelism

Splits operations within individual layers (e.g., attention mechanisms)

Enables fine-grained distribution of computational workloads

Zero Redundancy Optimizer (ZeRO)

Shards optimizer states and gradients across devices

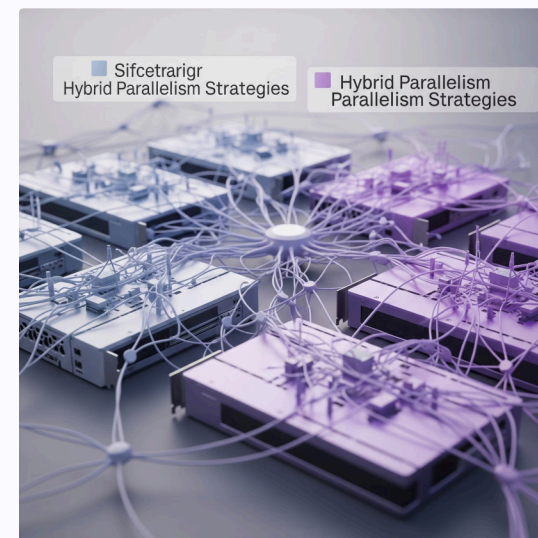
Dramatically reduces memory overhead without sacrificing model accuracy

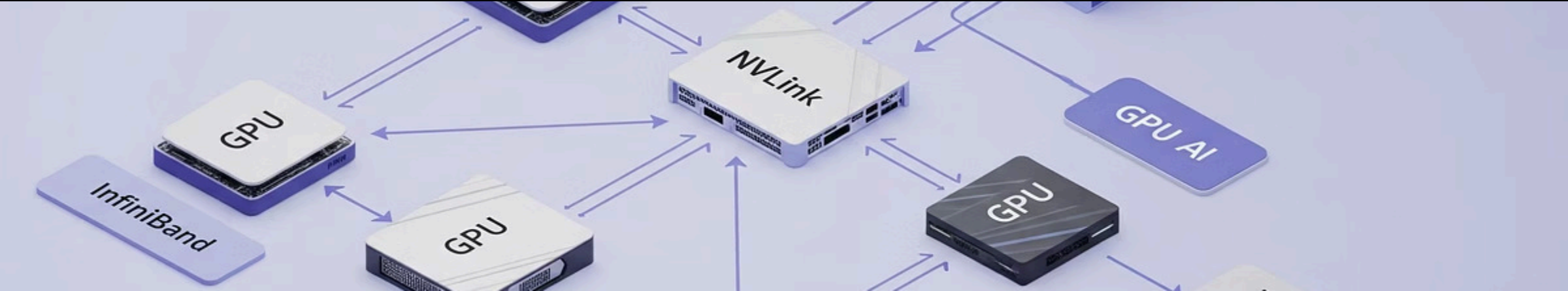
Fully Sharded Data Parallel (FSDP)

PyTorch implementation that scales beyond 100B parameters

Combines advantages of data parallelism with memory efficiency

📌 **Real-world applications:** State-of-the-art models like GPT and LLaMA implement hybrid parallelism strategies to achieve unprecedented scale





Communication Backbones

NCCL

NVIDIA Collective Communications Library

Optimized for all-reduce operations across NVIDIA GPUs

High-Speed Interconnects

InfiniBand / NVLink / NVSwitch provide critical low-latency connections

Essential for minimizing communication overhead

Ethernet

Common but becomes a significant bottleneck for large-scale clusters

Limited bandwidth impacts training efficiency

Training efficiency equation: Communication bandwidth vs. computation time ratio determines scalability limits

PyTorch DDP (Multi-GPU Single Node)

```
import torch
import torch.distributed as dist
from torch.nn.parallel import DistributedDataParallel as DDP

dist.init_process_group("nccl")
model = MyModel().to(local_rank)
model = DDP(model, device_ids=[local_rank])
```

Launch with:

```
torchrun --nproc_per_node=4 train.py
```

Key aspects:

- Each GPU receives a unique rank identifier
- Process group initialization configures communication
- Gradients automatically synchronized via all-reduce operations
- Minimal code changes from single-GPU training



The image shows two server racks in a data center. Each rack contains four NVIDIA GeForce RTX 4090 GPUs. The GPUs are connected to a network switch at the bottom of each rack via blue and green cables. The background is a blurred view of the server room with geometric light patterns.

PyTorch DDP (Multi-Node)

```
torchrun --nnodes=2 --nproc_per_node=4 \  
--rdzv_id=123 --rdzv_backend=c10d \  
--rdzv_endpoint=host0:29400 train.py
```

Configuration breakdown:

nnodes=2: Two machines participating in training

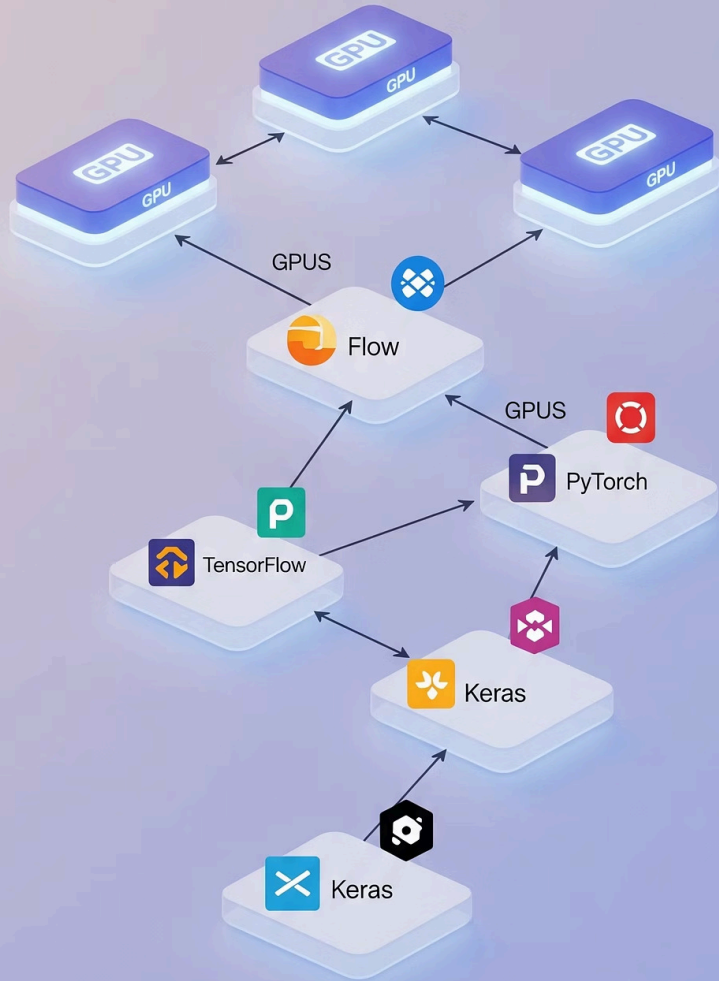
nproc_per_node=4: Four GPUs utilized on each machine

Total GPUs: 8 devices working together on the same training task

Rendezvous: Coordinates node discovery and process group formation

Horovod

Orchestrates Training deep is Frameworks



Horovod Example

```
import horovod.torch as hvd

hvd.init()
torch.cuda.set_device(hvd.local_rank())

optimizer =
hvd.DistributedOptimizer(
    optimizer,

named_parameters=model.named_parameters()
)
```

Advantages:

- Framework-agnostic API (works with TensorFlow, PyTorch, MXNet)
- Leverages MPI or NCCL as communication backend
- Minimal code changes required
- Developed at Uber, now widely adopted in industry

DeepSpeed & FSDP

DeepSpeed ZeRO

Stage 1

Shards optimizer states across GPUs

~4x memory reduction

Stage 2

Shards gradients + optimizer states

~8x memory reduction

Stage 3

Shards model parameters + gradients + optimizer states

Enables training models with trillions of parameters

PyTorch FSDP

Fully Sharded Data Parallel implementation in PyTorch:

- Combines parameter sharding with data parallelism
- Dynamically materializes parameters for forward/backward passes
- Native PyTorch API with minimal code changes
- Supports mixed precision training for additional efficiency

 Both technologies enable **trillion-parameter scale** models to be trained on relatively modest GPU clusters

Checkpointing & Fault Tolerance

Distributed Checkpointing

Each GPU writes its own rank-specific checkpoint shard

Coordinator ensures consistency across shards

Enables efficient resumption from arbitrary points

Checkpointing Frequency

Must increase with more GPUs due to higher failure probability

Especially critical when using preemptible/spot instances

Balance between I/O overhead and risk of lost computation

Resumption Requirements

Must reinitialize with identical world size and rank distribution

DDP/FSDP require consistent sharding patterns for recovery

Load balancers must account for recovery scenarios



Kubernetes + Multi-GPU Jobs

Using KubeFlow Training Operators (PyTorchJob, TFJob CRDs)

```
apiVersion: kubeflow.org/v1
kind: PyTorchJob
metadata:
  name: train-mgpu
spec:
  pytorchReplicaSpecs:
    Master:
      replicas: 1
      template:
        spec:
          containers:
            - image: my/train:1.0
          resources:
            limits:
              nvidia.com/gpu: 1
    Worker:
      replicas: 3
      template:
        spec:
          containers:
            - image: my/train:1.0
          resources:
            limits:
              nvidia.com/gpu: 1
```

Benefits:

- Declarative specification of distributed training
- Automatic pod placement and scheduling
- Health monitoring and restart capabilities
- Integration with GPU resource management
- Scales from single node to thousands of GPUs

Monitoring Training Efficiency

Key Metrics to Track



GPU Utilization

Target: >85% sustained utilization

Lower values indicate bottlenecks



Network Bandwidth

Monitor saturation during all-reduce

Critical for gradient synchronization



Step Time Variance

High variance indicates stragglers

Should remain within 5-10% across nodes

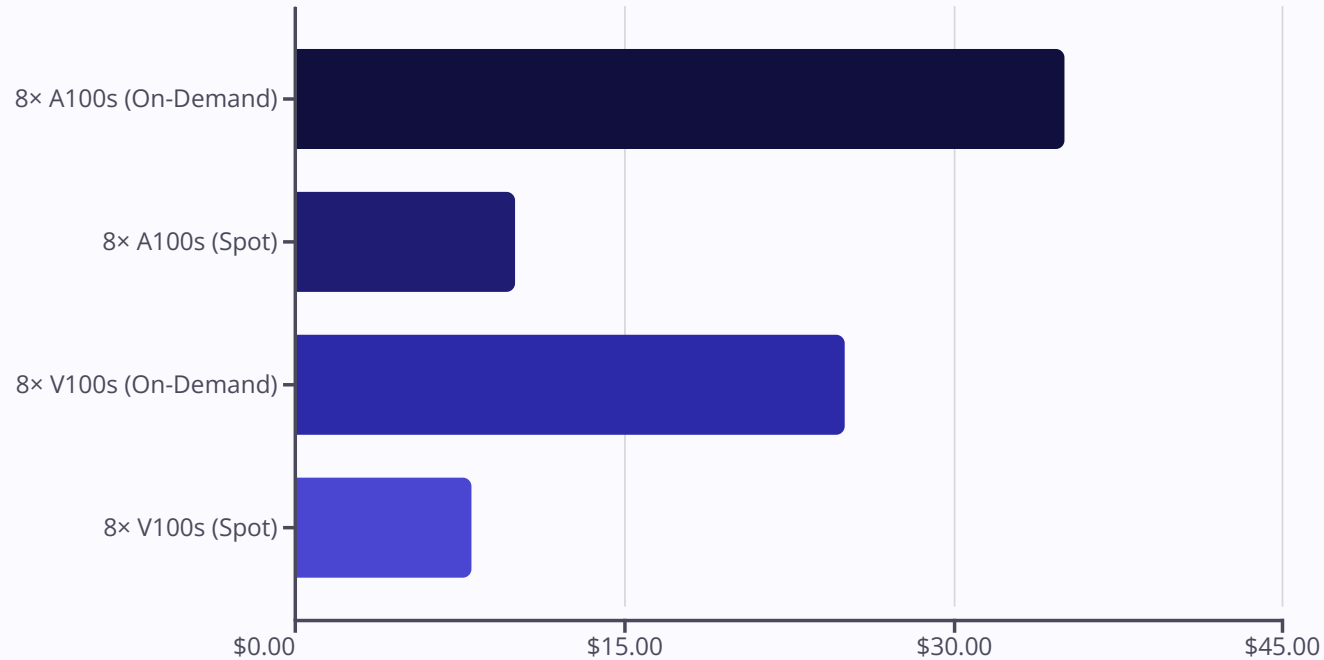
Monitoring Tools

- **NVIDIA DCGM:** Deep GPU telemetry
- **Prometheus:** Metrics collection
- **Grafana:** Real-time dashboards
- **WandB/MLflow:** Training metrics

Watch for stragglers! In multi-node jobs, a single slow GPU can bottleneck the entire training process



Cost Considerations



Cost Optimization Strategies

- Spot/preemptible instances save 60-80%
- Requires frequent checkpointing (every 15-30 min)
- Optimize batch size to maximize throughput
- Use mixed precision (FP16/BF16) to reduce memory and increase speed

ROI Considerations

Balance between:

- Time-to-result (business opportunity cost)
- Infrastructure cost (direct expense)
- Engineer productivity (development cycles)
- Final model quality (convergence issues)

Case Study: LLaMA Training

Meta's LLaMA models represent state-of-the-art in distributed training at scale

Architecture

Hybrid parallelism combining data, model, and tensor approaches

DeepSpeed ZeRO-3 for efficient optimizer state sharding

Scale

Trained across 2,000+ NVIDIA A100 GPUs

Months of continuous training at massive scale

Key Lessons

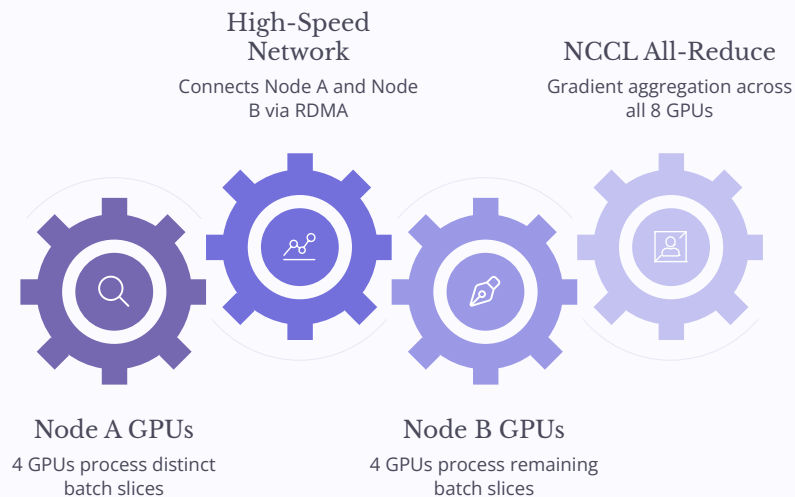
Communication fabric becomes the critical bottleneck at scale

Careful workload balancing required to maintain efficiency



Outcome: Produced a family of models from 7B to 65B parameters that match or exceed GPT-3 performance on many tasks while being much more efficient to deploy

Multi-Node Distributed Training



Multi-node DDP = scale out across GPUs with gradient synchronization via all-reduce operations

The diagram illustrates how distributed data parallel training coordinates across multiple nodes, with each GPU processing a slice of the batch and synchronizing gradients to ensure model consistency. The communication fabric between nodes is often the limiting factor for scaling efficiency.

Key Lessons for Multi-GPU Training

Choose the Right Parallelism Strategy

Select from data, model, pipeline, or hybrid approaches based on model architecture and available hardware

Optimize Communication Infrastructure

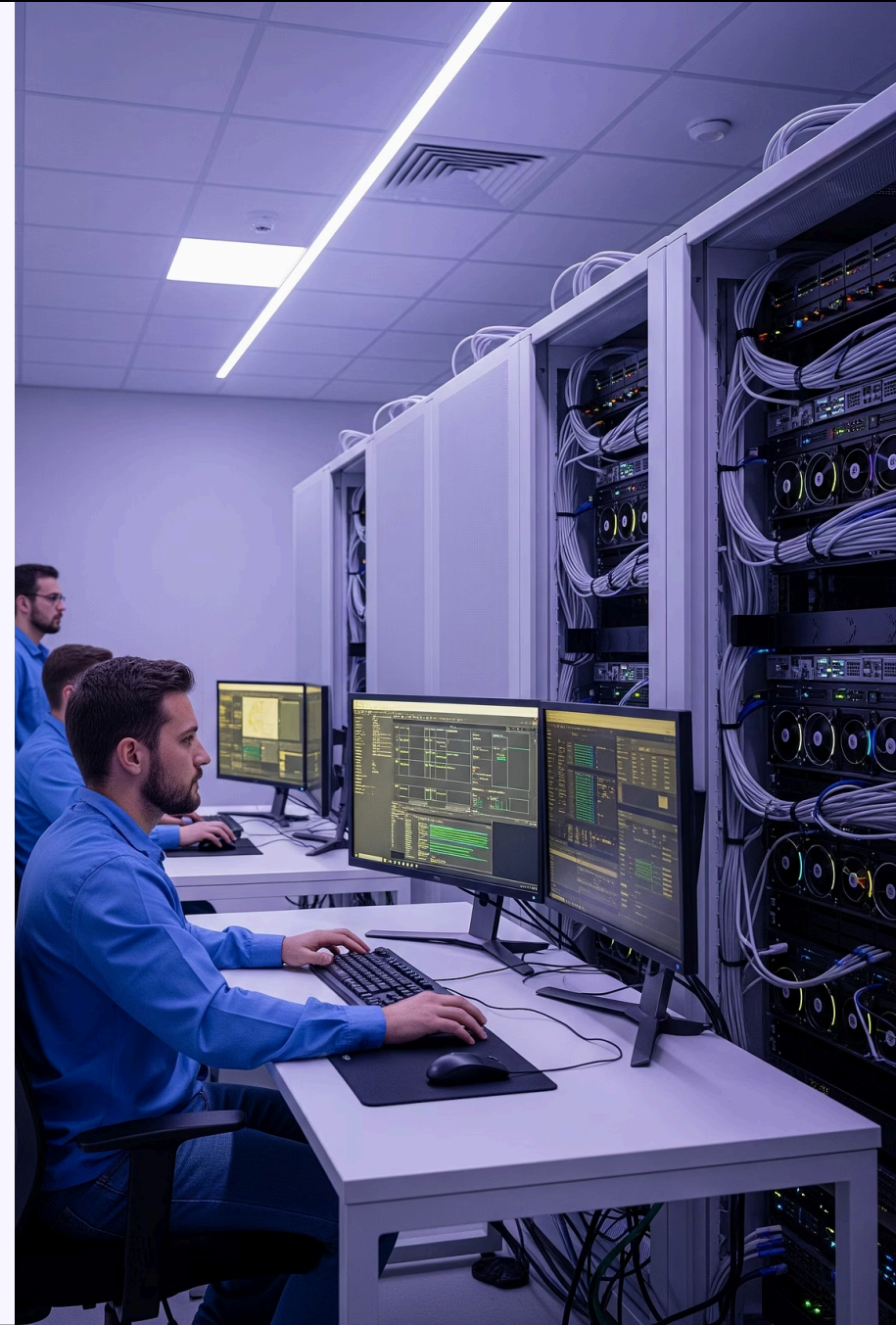
NCCL, InfiniBand, NVLink, and NVSwitch technologies are critical for efficient gradient synchronization and parameter sharing

Leverage Advanced Frameworks

DeepSpeed, FSDP, and similar tools enable training massive models by optimizing memory usage and communication patterns

Plan for Resilience and Cost

Implement robust checkpointing and consider the cost-efficiency tradeoffs when selecting hardware and training duration



Discussion Prompt

For your Capstone, would you prioritize fast time-to-train (more GPUs) or cost efficiency (fewer GPUs + longer runs)? Why?

Fast Time-to-Train Considerations

- Enables rapid experimentation cycles
- Reduces development iteration time
- Higher upfront cost, potentially lower total cost
- More complex infrastructure setup

Cost Efficiency Considerations

- Lower direct infrastructure expenses
- Simpler deployment and management
- Longer feedback loops for experiments
- Higher opportunity cost from delayed results

Consider your specific project constraints, model architecture, dataset size, and business requirements when making this decision.